

Cat Breed Classifier

Presented by

Lapasrada Sopanangkul 68070160

Presented to

Dr. Nat Dilokthanakul

This report is a part of 06046404

Fundamentals of Deep Learning Semester 1/2569

School of Information Technology

King Mongkut's Institute of Technology Ladkrabang

Preface

This report is about a design, implementation, and evaluation of a custom deep learning image classifier built to identify 12 cat breeds from the Oxford-IIIT Pet Dataset. Covering everything from data preparation and simple baseline models to experimental findings, visual confusion charts, and honest error analysis.

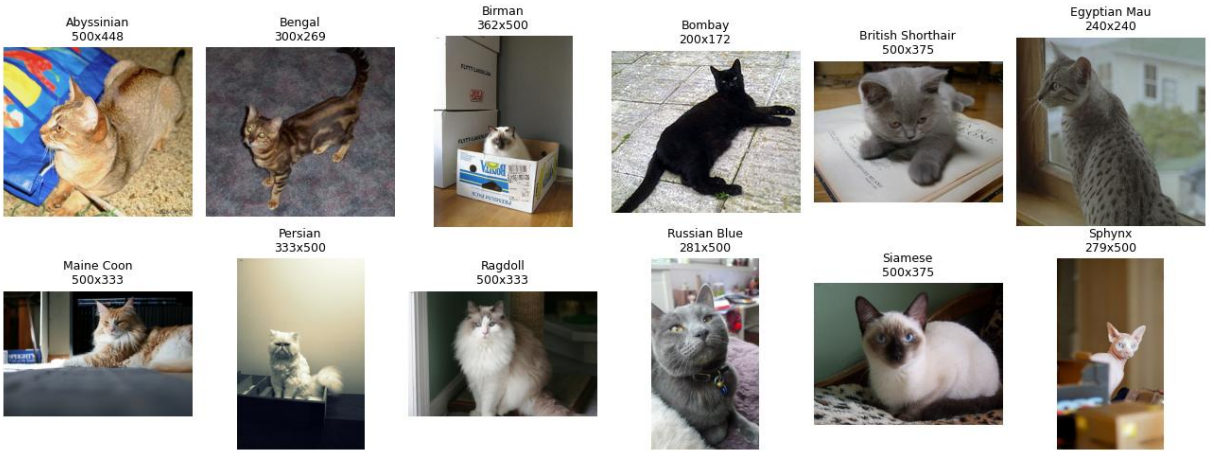
Lapasrada Sopanangkul

List of contents

Content	page
Preface	i
List of contents	ii
Dataset & task	1-2
Model & training choices	3
Baseline	4
Final test metric.	5-6
What I tried (ablation)	7-8
What worked / what failed	9-10
With more time	11
Reference	12

Dataset & task

Dataset: Oxford-IIIT Pet dataset (CC BY-SA 4.0), downloaded from the Hugging Face mirror timm/oxford-iiit-pet, keeping only the cat images.



Example of each cat breed's photo from data

```
  trainval: 1188 cat images
  test: 1183 cat images
train=950  val=238  test=1183   (no image shared between splits: OK)
breed                 train   val  test
Abyssinian               80    20    98
Bengal                   80    20   100
Birman                   80    20   100
Bombay                   77    19    88
British Shorthair        80    20   100
Egyptian Mau             74    19    97
Maine Coon               80    20   100
Persian                  80    20   100
Ragdoll                  80    20   100
Russian Blue             80    20   100
Siamese                  79    20   100
Sphynx                   80    20   100
decode check train: (950, 32, 32, 3) OK
decode check val: (238, 32, 32, 3) OK
decode check test: (1183, 32, 32, 3) OK
majority-class accuracy is about 8.4%  (chance = 8.3%)
```

Spilt: the cat's images (1,188 images) are split 80/20 per breed with a fixed seed of 42 into 950 training and 238 validation images. The official test cats (1,183 images) are kept untouched until the final evaluation. The code asserts that no image appears in two splits.

Preprocessing: each image is decoded once and resized to a 128×128 RGB, then normalized with the usual ImageNet mean and standard deviation, used here only as fixed constants. Augmentation is applied to the training set only with random resized crops, horizontal flip, rotation, mild brightness, contrast and saturation jitter. Hue is left alone because coat color identifies breeds.

Breed	Train	Val	Test
Abyssinian	80	20	98
Bengal	80	20	100
Birman	80	20	100
Bombay	77	19	88
British Shorthair	80	20	100
Egyptian Mau	74	19	97
Maine Coon	80	20	100
Persian	80	20	100
Ragdoll	80	20	100
Russian Blue	80	20	100
Siamese	79	20	100
Sphynx	80	20	100
Total	950	238	1,183

Class balance: the test set is close to balanced: about 100 images per breed (the row totals of the confusion matrix: Abyssinian 98, Bombay 88, Egyptian Mau 97, all others 100). That is why the majority-class baseline is near chance.

Task: the goal of this project was to build a model from scratch that can classify a picture of a cat and figure out which of 12 breeds it belongs to. Identifying cat breeds is hard for computers because cats of the same breed can look quite different (different fur colors, poses, background lighting), while cats of different breeds can look almost identical.

Model & training choices

Model: CatCNN is a plain four-block convolutional network with 1,176,300 trainable parameters, trained from random initialization with no pretrained weights.

```
input   (1, 3, 128, 128)
block1  (1, 32, 64, 64)   params    10,208
block2  (1, 64, 32, 32)   params    55,552
block3  (1, 128, 16, 16)   params   221,696
block4  (1, 256, 8, 8)   params   885,760
total trainable parameters: 1,176,300
```

- Stacked 3 × 3 convolutions cover a large area of the image with few weights.
- BatchNorm keeps training stable on a small dataset.
- Global average pooling replaces a large fully connected layer, which removes most of the parameters that would memorize the training photos.
- Channels double as the spatial size halves, so the network builds from edges and textures up to larger shapes.

Training choices: with under 1,000 training images the main risk is memorization, so the model uses augmentation, dropout, weight decay, label smoothing and best-validation-epoch checkpointing. Validation and test images are never augmented.

Setting	Value
Loss	Cross-entropy with label smoothing 0.1
Learning rate	2e-3, cosine decays to 0 over the run
Weight decay	5e-2
Batch size / epoch	32 / 60
Dropout	0.3 before the final linear layer.
Augmentation	Random resized crop (scale 0.6–1.0), horizontal flip, rotation ±10° and color jitter 0.2.
Checkpoint rule	keep the epoch with the best validation accuracy
Seed	42

Baseline

Two trivial baselines are:

1. Majority class: Always predict the most common training breed, which is Abyssinian. Test accuracy is 0.083 (8.3%) and validation 0.084. This is about chance because the classes are nearly balanced.

2. Logistic regression: on raw 32 × 32 color pixels (3,072 features, standardized, regularization strength C chosen on validation from 1e-4 to 1e-1). Test accuracy is 0.217 (21.7%) and validation 0.218. A linear model on tiny pixels gets roughly twice chance, so color alone carries a little breed information.

```
train=950 val=238 test=1183  classes=12
  logistic regression C=0.0001  val acc 0.218
  logistic regression C=0.001   val acc 0.210
  logistic regression C=0.01    val acc 0.202
  logistic regression C=0.1     val acc 0.197

baseline          val acc  test acc
majority            0.084     0.083   (always 'Abyssinian')
logreg_pixels       0.218     0.217
```

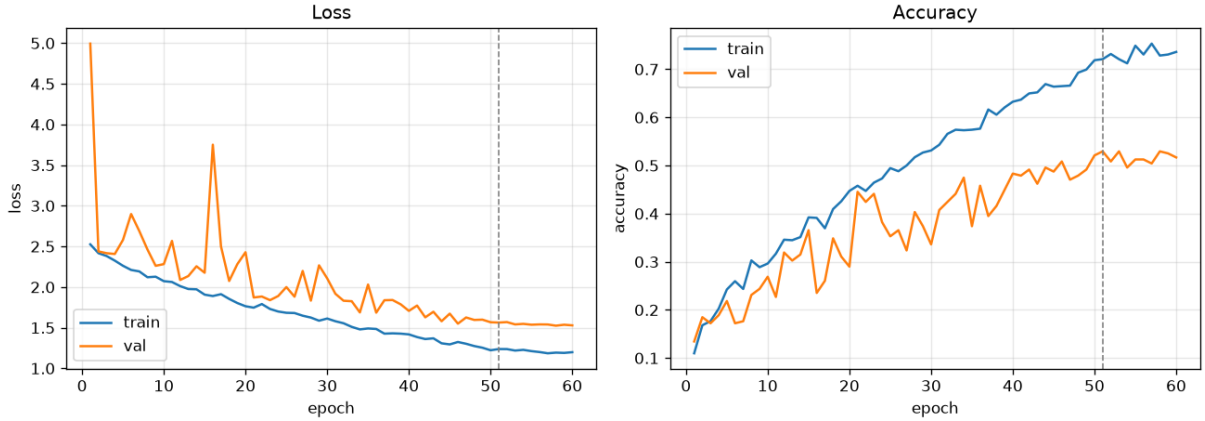
Final test metric

The test set was opened once, for one model:

Model	Test accuracy	Macro-F1
Majority class	0.083	0.013
Logistic regression on 32 × 32 pixels	0.217	0.19
CatCNN (CNN)	0.509 (95% interval 0.480–0.537)	0.498

Other test numbers for the CNN: top 3 accuracy 0.814, test loss 1.405, n = 1,183 images.

It beat the baseline clearly 50.9% against 8.3% for the majority class and 21.7% for logistic regression on pixels.

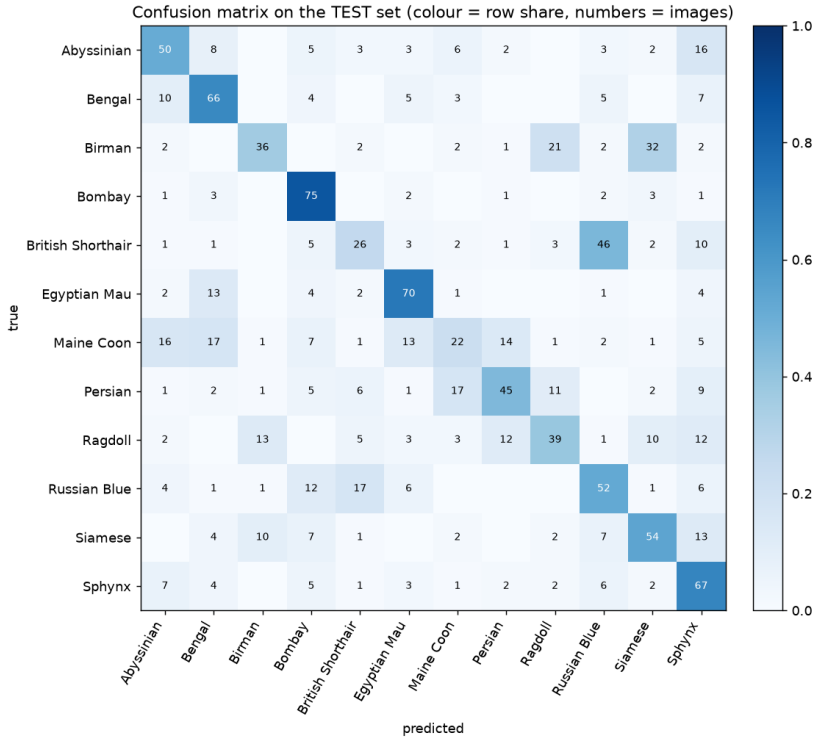


training curves of cnn model

Per-breed results, sorted by recall:

Breed	Test images	Recall	Precision	Macro- F1
Bombay	88	0.852	0.581	0.691
Egyptian Mau	97	0.722	0.642	0.680
Sphynx	100	0.670	0.441	0.532
Bengal	100	0.660	0.555	0.603
Siamese	100	0.540	0.495	0.517
Russian Blue	100	0.520	0.409	0.458
Abyssinian	98	0.510	0.521	0.515
Persian	100	0.450	0.577	0.506
Ragdoll	100	0.390	0.494	0.436
Birman	100	0.360	0.581	0.444
British Shorthair	100	0.260	0.406	0.317
Maine Coon	100	0.220	0.373	0.277

Breeds with a distinctive look do best (Bombay is all black, Sphynx is hairless, Egyptian Mau is spotted). Breeds that look alike or vary a lot between individuals do worst.



test confusion matrix

What I tried (ablation)

Run	Change vs. baseline	Best val acc	Train acc (same epoch)	Train-val gap	Best epoch
`cnn`	baseline	0.529	0.721	+0.191	51
`cnn_dropout05`	dropout=0.5	0.597	0.626	+0.029	43
`cnn_noaug`	no_aug=True	0.647	1.000	+0.353	56

Each run changes exactly one setting from the baseline run (cnn).

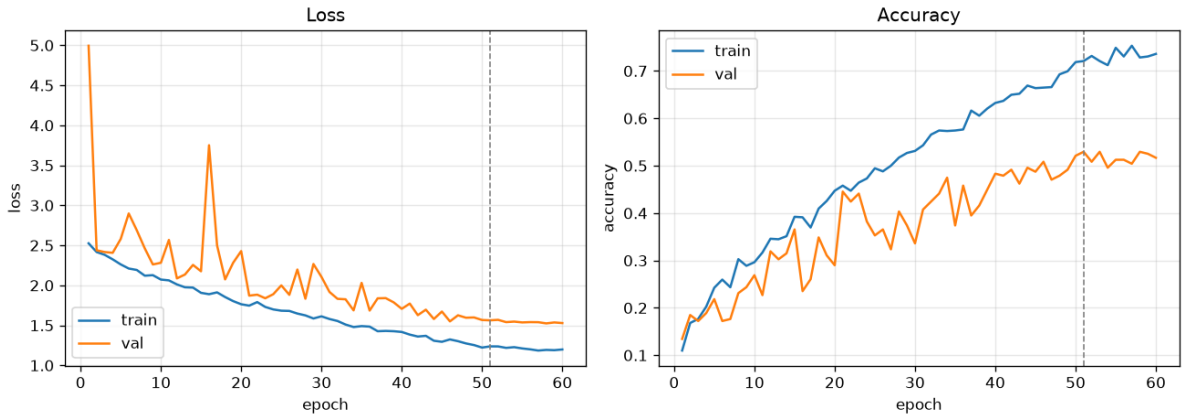
Removing augmentation gave the best validation accuracy of 0.647, +0.118 over cnn and dropout gave an accuracy of 0.597, +0.068 over cnn.

Images are squashed to a square and RandomResizedCrop then crops further into them. A crop can remove the face or coat pattern that identifies the breed, which makes the labels noisier. Milder augmentation or a longer schedule might change this result.

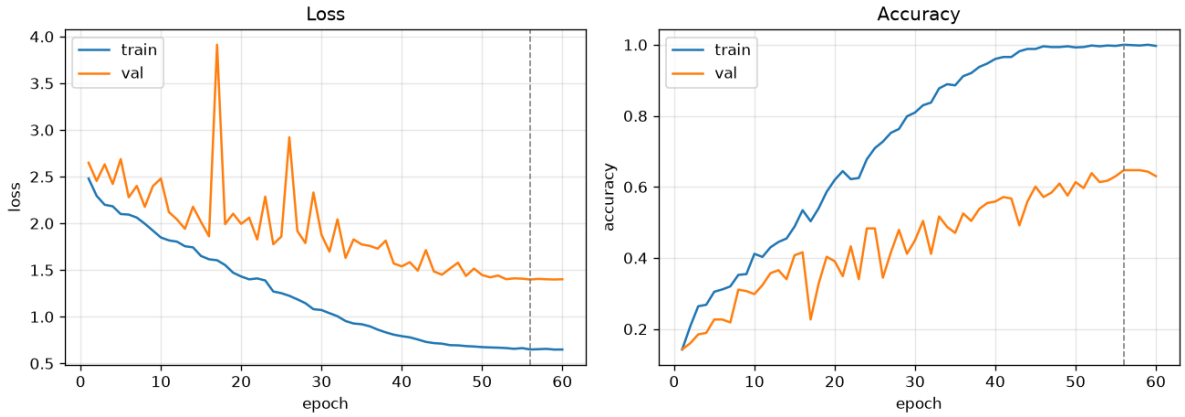
The train–val gap tells that:

- **cnn (augmentation on):** is under-fitting, training accuracy was only about 72–74% (measured on augmented images). Its lowest validation loss came at epoch 58 of 60, so it had not finished learning. Augmentation made the task harder, and 60 epochs did not make it better.
- **cnn_dropout05:** had smallest gap (+0.03), but also lower training accuracy. The small gap comes from learning less, not from generalizing better. Its validation loss was still at its lowest at epoch 60.
- **cnn_noaug:** is strong over-fitting, training accuracy reached 100% while validation stayed near 65%, a gap of 35 points. The network memorized the training photos. It still scored best on validation, so memorizing unaugmented data beats learning slowly with heavy augmentation.

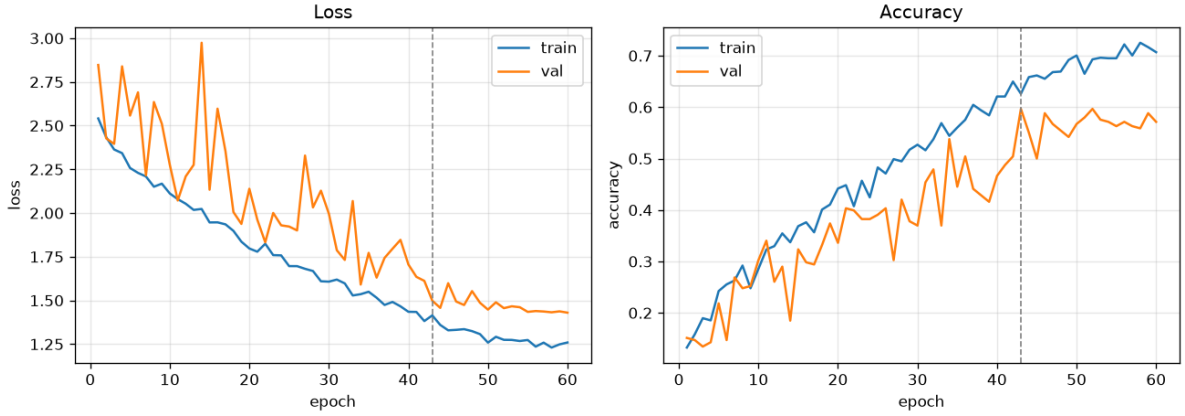
Training curves of each model:



training curves of cnn model



training curves of cnn_noaug model



training curves of cnn_dropout05 model

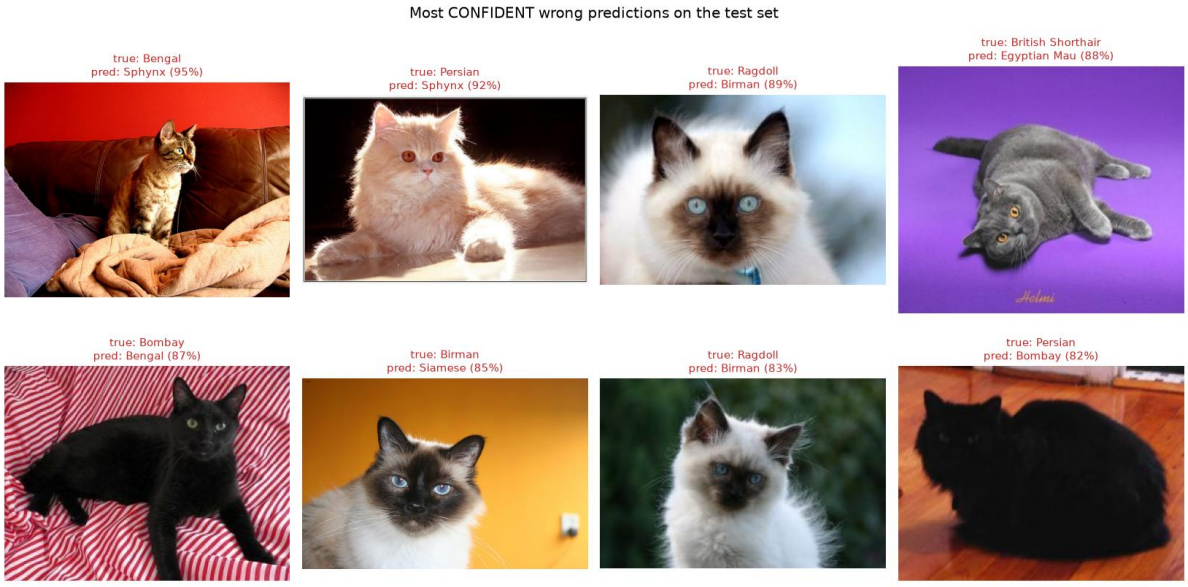
What worked / what failed

What worked:

- The CNN is far above both chance and the majority baseline, and a top 3 accuracy of 81% shows it learned real breed features.
- Breeds with a distinctive coat are recognized best such as Bombay (all black), Egyptian Mau (spotted), Sphynx (hairless) and Bengal. That fits a model that relies mostly on coat color and texture.
- Maine Coon is the weakest breed to predict, and its errors are spread widely as Bengal 17, Abyssinian 16, Persian 14 and Egyptian Mau 13. There is no look-alike, which suggests the model never found features for this breed and is mostly guessing.

What failed: British Shorthair vs. Russian Blue and Birman vs. Siamese/Ragdoll.

On the test set, 46 British Shorthair photos were predicted as Russian Blue (British Shorthair recall is only 0.26), while 17 Russian Blues predicted as British Shorthair. Both breeds are short-haired grey-blue cats with round faces, so color alone does not separate them. 32 Birman images were predicted as Siamese and 21 as Ragdoll. Birman, Ragdoll and Siamese all have pale bodies with dark face points and often blue eyes.



True breed	Predicted	Confidence	What the photo shows
Bengal	Sphynx	95%	Tabby cat sitting on a brown leather sofa, red wall behind
Persian	Sphynx	92%	Cream long-haired cat in strong light on a dark background
Ragdoll	Birman	89%	Pale cat with dark face and blue eyes
British Shorthair	Egyptian Mau	88%	Grey cat lying on a purple backdrop
Bombay	Bengal	87%	Black cat on red-and-white striped cloth
Birman	Siamese	85%	Pale cat with dark face and blue eyes, orange wall
Ragdoll	Birman	83%	White kitten with dark face and blue eyes
Persian	Bombay	82%	Black long-haired cat on a wooden floor

The eight most confident wrong predictions (chosen automatically by confidence) show that:

- Two Ragdolls and a Birman were mistaken for Birman or Siamese at 83–89% confidence. These are genuinely hard pairs.
- A cream long-haired Persian, backlit against a dark background, was called Sphynx at 92%. A very dark fluffy cat labelled Persian was called Bombay at 82%.
- A black Bombay lying on a red-and-white striped cloth was called Bengal at 87%. A grey British Shorthair on a bright purple background was called Egyptian Mau at 88%.

With more time

With more time I would try to:

1. train the augmented model for much longer (120+ epochs): since it was still improving at epoch 60. I would change only the epoch count and keep every other setting and judge it on validation only. The ablation points to under-fitting in the augmented run is its training accuracy was only about 72%, and its lowest validation loss came at epoch 58 of 60, so it was still learning when the cosine schedule ended.

2. Resize the short side and center-crop: instead of squashing the photo into a square, so faces and coats are not distorted.

3. Milder augmentation: Use a crop scale of 0.8–1.0 with flip only, since the strong crops may remove the face or coat pattern that identifies the breed.

4. Larger input or wider network: Try 160 × 160 images or width 48 to give the model finer coat and face detail.

5. Target the look-alike pairs: British Shorthair vs. Russian Blue and Birman vs. Siamese/Ragdoll cause most of the errors, so inspect those images and consider extra care for them (for example, higher loss weight for those breeds.

6. Collect more photos: More labelled cats per breed, because only about 79 training images per breed is might not enough.

References

- Oxford-IIIT Pet dataset, CC BY-SA 4.0: https://www.robots.ox.ac.uk/~vgg/data/pets/
- Hugging Face mirror: https://huggingface.co/datasets/timm/oxford-iiit-pet